

```
===== Python Code =====
import numpy as np
from PIL import Image

img = Image.open("einstein.jpg").convert("L")
ImJPG = np.array(img)

m, n = ImJPG.shape

print("Shape:", ImJPG.shape)
print("m =", m)
print("n =", n)
print("Data type:", ImJPG.dtype)
print("Min:", ImJPG.min())
print("Max:", ImJPG.max())

import matplotlib.pyplot as plt

plt.imshow(ImJPG, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.savefig("output_step2.png", bbox_inches='tight')

ImJPG_center = ImJPG[99:m-100, 99:n-70]

print("Center shape:", ImJPG_center.shape)

plt.figure()
plt.imshow(ImJPG_center, cmap='gray')
plt.title("Center Crop")
plt.axis('off')
plt.savefig("output_step3_center.png", bbox_inches='tight')

# Create a black image (same size as original)
ImJPG_black = np.zeros((m, n), dtype=np.uint8)

# Get center dimensions
mc, nc = ImJPG_center.shape

# Compute where to place it (centered)
start_row = (m - mc) // 2
start_col = (n - nc) // 2

# Paste the cropped image into the black image
ImJPG_black[start_row:start_row+mc, start_col:start_col+nc] = ImJPG_center

# Save result
plt.figure()
plt.imshow(ImJPG_black, cmap='gray')
plt.title("Cropped Image on Black Background")
plt.axis('off')
plt.savefig("output_step4_black.png", bbox_inches='tight')

ImJPG_vertflip = np.flipud(ImJPG)

plt.figure()
plt.imshow(ImJPG_vertflip, cmap='gray')
plt.title("Vertical Flip")
plt.axis('off')
plt.savefig("output_step5_vertflip.png", bbox_inches='tight')

ImJPG_transpose = ImJPG.T

plt.figure()
plt.imshow(ImJPG_transpose, cmap='gray')
plt.title("Transpose")
plt.axis('off')
```

```
plt.savefig("output_step6_transpose.png", bbox_inches='tight')

ImJPG_horzflip = np.fliplr(ImJPG)

plt.figure()
plt.imshow(ImJPG_horzflip, cmap='gray')
plt.title("Horizontal Flip")
plt.axis('off')
plt.savefig("output_step7_horzflip.png", bbox_inches='tight')

ImJPG_rot90 = np.rot90(ImJPG)

plt.figure()
plt.imshow(ImJPG_rot90, cmap='gray')
plt.title("Rotate 90 Degrees Counterclockwise")
plt.axis('off')
plt.savefig("output_step8_rot90.png", bbox_inches='tight')

ImJPG_negative = 255 - ImJPG

plt.figure()
plt.imshow(ImJPG_negative, cmap='gray')
plt.title("Negative Image")
plt.axis('off')
plt.savefig("output_step9_negative.png", bbox_inches='tight')

# Darken (scale down)
ImJPG_dark = (0.5 * ImJPG).astype(np.uint8)

# Lighten (scale up, but cap at 255)
ImJPG_light = np.clip(1.5 * ImJPG, 0, 255).astype(np.uint8)

# Save both
plt.figure()
plt.imshow(ImJPG_dark, cmap='gray')
plt.title("Darkened Image")
plt.axis('off')
plt.savefig("output_step10_dark.png", bbox_inches='tight')

plt.figure()
plt.imshow(ImJPG_light, cmap='gray')
plt.title("Lightened Image")
plt.axis('off')
plt.savefig("output_step10_light.png", bbox_inches='tight')

# Build the four panels the way the project specifies
top_left = ImJPG
top_right = np.clip(ImJPG.astype(np.int16) - 50, 0, 255).astype(np.uint8)
bottom_left = np.clip(ImJPG.astype(np.int16) + 100, 0, 255).astype(np.uint8)
bottom_right = np.clip(ImJPG.astype(np.int16) + 50, 0, 255).astype(np.uint8)

# Concatenate into a 2x2 block image
top_row = np.hstack((top_left, top_right))
bottom_row = np.hstack((bottom_left, bottom_right))
ImJPG_Warhol = np.vstack((top_row, bottom_row))

plt.figure()
plt.imshow(ImJPG_Warhol, cmap='gray')
plt.title("Warhol Style Composition")
plt.axis('off')
plt.savefig("output_step11_warhol.png", bbox_inches='tight')

ImJPG_bw = (255 * np.floor(ImJPG / 128)).astype(np.uint8)

plt.figure()
plt.imshow(ImJPG_bw, cmap='gray')
plt.title("Black and White")
plt.axis('off')
```

```
plt.savefig("output_step12_bw.png", bbox_inches='tight')

ImJPG8 = (np.round(ImJPG / 255 * 7) * (255 / 7)).astype(np.uint8)

plt.figure()
plt.imshow(ImJPG8, cmap='gray')
plt.title("8 Gray Levels")
plt.axis('off')
plt.savefig("output_step13_8gray.png", bbox_inches='tight')

ImJPG_highcontrast = np.where(ImJPG < 128, 0, 255).astype(np.uint8)

plt.figure()
plt.imshow(ImJPG_highcontrast, cmap='gray')
plt.title("High Contrast Image")
plt.axis('off')
plt.savefig("output_step14_highcontrast.png", bbox_inches='tight')

gamma = 0.5 # try different values later

ImJPG_gamma = (255 * (ImJPG / 255) ** gamma).astype(np.uint8)

plt.figure()
plt.imshow(ImJPG_gamma, cmap='gray')
plt.title(f"Gamma Correction (gamma={gamma})")
plt.axis('off')
plt.savefig("output_step15_gamma.png", bbox_inches='tight')

===== Program Output =====
Shape: (433, 325)
m = 433
n = 325
Data type: uint8
Min: 0
Max: 245
Shape: (433, 325)
m = 433
n = 325
Data type: uint8
Min: 0
Max: 245
Center shape: (234, 156)
```